Asynchronous Replication of Configuration Tables in a Distributed, Multi-Tenant Database Topology

Alexandro Oliveira (alexandro.oliveira@cs.cruzeirodosul.edu.br)

Abstract. This work models a data-replication problem in a distributed environment. A source database holds system-wide configuration data that must be available in every regional database so that each node can operate independently. We describe the topology, the consistency model, and the constraints under which replication must hold, and we discuss the open design question of how to handle deletions that may leave orphaned tenant-referencing rows. The system targets a eventual consistency model with a uniform schema across all nodes and asynchronous propagation from the source of truth.

1 Introduction

In a monolithic architecture, where a single database instance serves the entire system, data replication is a concern. PostgreSQL, for example, allows tenant isolation through separate databases (logical database within the same instance), or schemas.

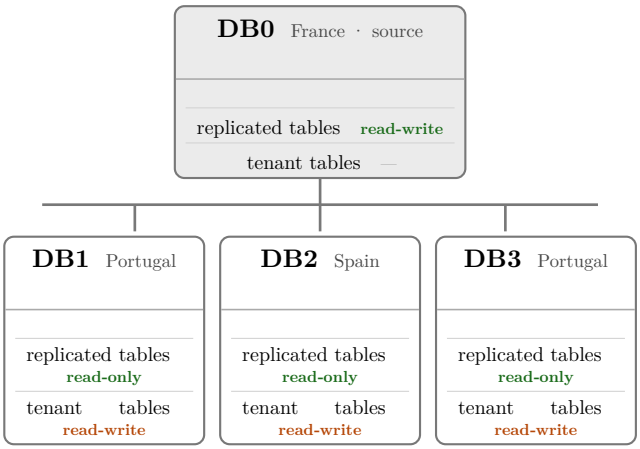
Another challenge in asynchronous data replication involves maintaining referential integrity when replicated tables contain foreign key relationships. Update and delete operations may lead to temporary inconsistencies across distributed databases due to replication delays, making it difficult to enforce foreign key constraints consistently in a microservices environment [1].

This research utilizes an experimental case study methodology. The experimental artifacts and services developed for this study will be implemented using Ruby on Rails and event-driven architectural principles, enabling the reproduction of realistic microservices scenarios and the analysis of different replication strategies and architectural alternatives.

The results of this study will provide insights and proof-of-concept are presented in this work.

2 Context

We maintain a database in France that stores general information relevant to the system setup and configuration: users, manufacturers, suppliers, and similar shared entities. Each regional database must hold a **copy** of this information so that the system operates in a distributed and independent fashion. In addition to the replicated configuration data, each node stores the raw and analytical data belonging to its own tenant.



Only replicated tables flow from DB0; tenant tables stay read-write locally.

Figure 1: Replication topology: the source database (DB0) propagates configuration data to the regional databases, each of which also holds its own tenant data.

3 System Model

For brevity we refer to the participating databases as follows:

Node	Role
DB0	France, source of truth for replicated data
DB1, DB3	Located physically Portugal
DB2	Located physically Spain

DB0 is the single writer for the replicated configuration tables; all other nodes are read replicas with respect to that data, while remaining full owners of their own tenant-specific data.

A clear ownership boundary separates the two classes of data on every node:

Table class	On DB0	On DB1, DB2, DB3
Replicated (configuration)	read-write	read-only
Tenant	n/a	read-write

The replicated tables are **never written directly** on a tenant node. The only party that may modify them is DB0, which then propagates the change downstream; on DB1, DB2, and DB3 these tables are strictly read-only. Conversely, the tenant-owned tables are read-write only on the node that owns them, and are not subject to replication.

4 Constraints and Boundaries

The replication mechanism operates under the following constraints:

1. **Weak consistency.** We do not need to guarantee that the data is synchronized across all databases at the same instant; we only need to guarantee that every database eventually holds a copy of the tenant's data.
2. **Uniform schema.** The schema must be uniform across all databases so that replication works correctly.
3. **Tenant-referencing foreign keys.** Tables considered replicable may carry foreign keys into tenant data. For example, a replicated `Record` table may have a foreign key into a tenant-owned `Tenant` table.
4. **Asynchronous synchronization.** When a row is inserted or modified in DB0, the change must be propagated to the other databases asynchronously.
5. **Single writer for replicated data.** Replicated tables are written exclusively on DB0; on the tenant nodes (DB1, DB2, DB3) they are read-only. No tenant node may write directly to a replicated table; all changes originate at DB0 and flow downstream. Only the tenant-owned tables are read-write, and only on their owning node.

5 (DRAFT) Future Work

A central open question concerns deletions. When a row is deleted in DB0, the same row should be deleted in DB1, DB2, and DB3. If that row is referenced by a foreign key on one of the tenants, several strategies are possible:

- **Prevent** the deletion while references exist;
- Perform a **distributed transaction** across the affected nodes; or
- Delete the row anyway and **tolerate orphaned** tenant-referencing data.

References

[1] S. Kanwal, N. R. Chaudhry, R. Choudhary, Y. A. Shaik, P. Yadav, and A. Rashid, "Foreign Key Constraints to Maintain Referential Integrity in Distributed Database in Microservices Architecture," *International Journal of Advanced Computer Science and Applications*, vol. 16, no. 6, pp. 974–985, 2025, doi: 10.14569/IJACSA.2025.0160696.